

Полиномиальные алгоритмы на графах: от обходов до потоков минимальной стоимости — анализ для аспирантского экзамена

Обходы Графов: Поиск в Глубину и Ширину

Обходы графов являются фундаментальными процедурами, лежащими в основе многих более сложных алгоритмов в теории графов. Цель этих процедур — систематически посетить все вершины и рёбра связного графа или каждой компоненты связности в несвязном графе [88](#). Наиболее известными и широко используемыми методами являются поиск в ширину и поиск в глубину. Несмотря на общую цель, они достигают её с помощью принципиально различных стратегий, что определяет их уникальные свойства и области применения. Поиск в ширину исследует граф "уровень за уровнем", начиная с источника, и гарантирует нахождение кратчайшего пути в невзвешенном графе, тогда как поиск в глубину погружается в граф как можно дальше по каждому пути, прежде чем вернуться и исследовать другие ветви [17](#) [18](#).

Поиск в ширину (Breadth-First Search, BFS) начинается с выбора начальной вершины s . Алгоритм просматривает все вершины, смежные с s , затем все вершины, смежные с этими вершинами (кроме s), и так далее, покрывая граф слоями, где каждый слой состоит из вершин, находящихся на одинаковом расстоянии от исходной вершины s [18](#). Для реализации этой стратегии используется очередь. Изначально все вершины помечаются как "не посещенные". Вершина s помещается в очередь и помечается как посещенная. Затем начинается цикл: пока очередь не пуста, извлекается вершина u , и для каждой ее непосещенной соседней вершины v выполняются действия: v помечается как посещенная, устанавливается предок v как u (что позволяет восстановить путь), и v помещается в конец очереди [10](#) [166](#). Этот процесс продолжается до тех пор, пока все доступные для обхода вершины не будут рассмотрены. Важнейшим свойством BFS является то, что он находит кратчайший путь (в смысле числа ребер) от начальной вершины s до любой

достижимой вершины v ¹⁴². Это следует из того, что вершины добавляются в очередь в порядке их возрастания расстояния от s . Временная сложность алгоритма составляет $O(V+E)$, где V — количество вершин, а E — количество рёбер. Это связано с тем, что каждая вершина и каждое ребро рассматриваются не более одного раза ⁶⁰. При использовании списка смежности пространственная сложность также равна $O(V+E)$, поскольку в худшем случае очередь может содержать все вершины ^{21 22}.

Поиск в глубину (Depth-First Search, DFS) работает по иному принципу. Он начинается с вершины s и рекурсивно или итеративно исследует один путь до тех пор, пока не достигнет тупика (вершины, у которой нет непосещенных соседей). В этот момент алгоритм "откатывается" назад на предыдущую вершину и проверяет, есть ли у неё другие непосещенные соседи. Если да, он продолжает обход по новому пути. Этот процесс повторяется, пока не будут посещены все вершины, достижимые из s ¹⁷. Реализация DFS может быть рекурсивной, где вызов функции для соседней вершины помещает её на системный стек, или итеративной с явным использованием стека ¹⁷⁷. Алгоритм выглядит следующим образом: все вершины помечаются как "не посещенные". Вызывается функция **DFS-VISIT(s)**. Эта функция помечает текущую вершину u как "посещаемую", затем для каждой смежной с u вершины v , если v "не посещена", устанавливается предок v как u и вызывается **DFS-VISIT(v)** ^{89 107}. DFS часто используется для задач, таких как определение наличия циклов в графе (если при обходе найдена уже посещенная вершина, которая не является родителем текущей), топологической сортировки и поиска компонент связности. Его временная сложность также равна $O(V+E)$, так как каждая вершина и каждое ребро рассматриваются один раз ⁶⁰. Однако пространственная сложность в худшем случае может достигать $O(V)$ из-за глубины рекурсии или размера стека, что соответствует длинному пути в дереве ²².

Характеристика	Поиск в ширину (BFS)	Поиск в глубину (DFS)
Стратегия	Исследование уровня за уровнем 18	Погружение вглубь, откатка 17
Используемая структура данных	Очередь 10	Стек (системный или явный) 61
Гарантия кратчайшего пути	Да, в невзвешенном графе 142	Нет
Временная сложность	$O(V+E)$ 21	$O(V+E)$ 60
Пространственная сложность	$O(V+E)$ 22	$O(V)$ 22
Основное применение	Поиск кратчайшего пути, уровень за уровнем 18	Топологическая сортировка, поиск циклов 178

Выбор между BFS и DFS во многом зависит от представления графа. Для разреженных графов, где число рёбер E значительно меньше V^2 , списки смежности являются предпочтительным способом представления, так как они экономят память и позволяют быстро перебирать соседей вершины [23 124](#). При использовании списка смежности время доступа к конкретному ребру (u,v) может быть линейным по числу соседей u , но общая сумма всех операций по перебору всех списков смежности составит $O(E)$. В то же время, для плотных графов, где E близко к V^2 , матрица смежности может быть более удобной, так как проверка существования ребра (u,v) занимает постоянное время $O(1)$ [19](#). Однако она требует $O(V^2)$ памяти, что может быть избыточным. Современные подходы могут комбинировать эти структуры, например, используя хеш-таблицы для списка смежности, чтобы получить быстрый доступ к соседям, сохраняя при этом преимущества списков смежности по памяти [84 125](#). Для большинства практических задач на графах со списками смежности, как BFS, так и DFS имеют одинаковую асимптотическую сложность, и выбор между ними определяется характером решаемой задачи: если важна кратчайность пути, выбирается BFS; если требуется исследовать все возможные пути или найти структурные особенности графа, такой как циклы или мосты, предпочтительнее DFS [179](#).

Алгоритмы для Минимального Остовного Древа: Прим и Крускала

Задача нахождения минимального остовного дерева является одной из классических задач комбинаторной оптимизации на графах. Она заключается

в нахождении такого подграфа данного связного, неориентированного и взвешенного графа, который является деревом, содержит все вершины исходного графа и имеет минимальную сумму весов своих рёбер [29](#). Такие деревья играют ключевую роль в проектировании сетей (например, телефонных, электрических или компьютерных), где необходимо соединить все точки с минимальными затратами [82](#). Два наиболее известных и широко используемых алгоритма для решения этой задачи — это алгоритм Прима и алгоритм Крускала. Оба они являются классическими примерами жадных алгоритмов, однако они реализуют жадную стратегию по-разному [70 156](#).

Алгоритм Крускала строит MST путем последовательного добавления рёбер в порядке возрастания их веса. Идея заключается в том, чтобы всегда выбирать самое легкое доступное ребро, которое не образует цикл с уже выбранными ранее рёбрами [34](#). Процесс начинается с того, что все рёбра графа сортируются по их весу. Затем алгоритм итеративно рассматривает рёбра в порядке от самого легкого к самому тяжелому. Для каждой пары вершин, соединяемых ребром, проверяется, принадлежат ли они к одной и той же компоненте связности. Если нет, это ребро добавляется к MST, и две компоненты объединяются. Если да, ребро пропускается, так как его добавление создало бы цикл. Для эффективной проверки принадлежности к компоненте связности и объединения множеств используется структура данных «Система непересекающихся множеств» (Union-Find) [35 37](#). Сложность алгоритма Крускала определяется в основном временем на сортировку рёбер, которое составляет $O(E \log E)$ или, что эквивалентно, $O(E \log V)$, где E — число рёбер, а V — число вершин [33 45](#). Поскольку в каждом шаге операции Find и Union выполняются почти за константное время (обратная функция Аккермана), они не доминируют над сложностью. Таким образом, итоговая временная сложность алгоритма Крускала — $O(E \log V)$ [47](#). Пространственная сложность составляет $O(V+E)$ для хранения графа и структуры Union-Find [72](#).

Алгоритм Прима, в свою очередь, строит MST постепенно, начиная с одной единственной вершины и расширяя полученное дерево на одну вершину за раз [70](#). На каждом шаге алгоритм находит самое легкое ребро, которое соединяет вершину, принадлежащую уже построенному дереву, с вершиной, которая еще не входит в него. Это ребро и соответствующая ему вершина добавляются к дереву. Чтобы эффективно находить такое ребро, используется приоритетная очередь (или "мин-куча"). В этой очереди хранятся все вершины, еще не входящие в MST, отсортированные по минимальному весу ребра, соединяющего их с уже построенным деревом. Когда вершина добавляется к MST, алгоритм

обновляет ключи (веса) всех её соседей в очереди, если найдено более легкое соединение с деревом [28 207](#). Изначально в приоритетной очереди находится одна произвольная вершина с ключом 0, а все остальные — с бесконечным ключом. Алгоритм продолжается до тех пор, пока все вершины не будут добавлены в MST. Сложность алгоритма Прима сильно зависит от реализации приоритетной очереди. При использовании стандартной бинарной кучи сложность составляет $O(E \log V)$ [205](#). Более продвинутой реализацией с использованием Fibonacci-кучи позволяет достичь временной сложности $O(E + V \log V)$ [71 196](#). Пространственная сложность также составляет $O(V + E)$ [72](#). В целом, выбор между двумя алгоритмами зависит от структуры графа: для разреженных графов, где E мало, алгоритм Крускала, чья сложность зависит от числа рёбер, может быть предпочтительнее. Для плотных графов, где E близко к V^2 , алгоритм Прима может работать быстрее [46 82](#).

Характеристика	Алгоритм Крускала	Алгоритм Прима
Жадная стратегия	Рассматривать рёбра в порядке возрастания веса 34	Расширять дерево от одной вершины, добавляя самые лёгкие рёбра 70
Основная структура данных	Сортировка рёбер, система непересекающихся множеств (Union-Find) 35	Приоритетная очередь (мин-куча) 28
Временная сложность	$O(E \log E)$ или $O(E \log V)$ 33	$O(E \log V)$ (с бинарной кучей) или $O(E + V \log V)$ (с Фибоначчиевой кучей) 71
Пространственная сложность	$O(V + E)$ 72	$O(V + E)$ 72
Начальный объект	Все рёбра графа	Одна вершина

Поиск Кратчайших Путь: Дейкстры и A*

Задача нахождения кратчайших путей является одной из центральных в теории графов и имеет множество приложений, от навигационных систем до планирования маршрутов в компьютерных сетях [25 152](#). Классический случай, решаемый алгоритмом Дейкстры, заключается в поиске кратчайших путей от одной заданной исходной вершины до всех остальных вершин в графе, веса всех рёбер которого неотрицательны [39 150](#). Идея алгоритма основана на жадной стратегии и называется "релаксацией". Алгоритм поддерживает для каждой вершины оценку кратчайшего пути из исходной вершины. Изначально эта оценка равна нулю для исходной вершины и бесконечности для всех

остальных. На каждом шаге алгоритм выбирает вершину u , для которой оценка кратчайшего пути является минимальной среди всех вершин, которые ещё не были окончательно обработаны. Затем он "релаксирует" все рёбра, исходящие из u . Релаксация ребра (u, v) с весом $w(u, v)$ означает проверку, не даёт ли путь через u лучшую (меньшую) оценку для вершины v . Если да, оценка для v обновляется до значения $\text{dist}[u] + w(u, v)$. Этот процесс повторяется до тех пор, пока в графе остаются необработанные вершины 101170. Для эффективного поиска вершины с минимальной оценкой используется приоритетная очередь (мин-куча), которая хранит пары (оценка_расстояния, вершина). Сложность алгоритма Дейкстры при использовании бинарной кучи составляет $O(E \log V)$ 205. Эта сложность складывается из $O(E)$ операций релаксации и $O(V)$ извлечений минимума из очереди, каждая из которых занимает $O(\log V)$ времени. Алгоритм гарантирует нахождение точного решения благодаря тому, что веса рёбер неотрицательны: как только вершина извлекается из очереди, её кратчайшее расстояние окончательно установлено и никогда не будет изменено 203.

Хотя алгоритм Дейкстры очень эффективен, он может быть неоптимальным в ситуациях, когда нас интересует кратчайший путь только до одной конкретной целевой вершины. В таких случаях мы можем направить поиск "в сторону" цели. Именно эту идею реализует семейство эвристических алгоритмов, наиболее известным из которых является алгоритм А. Алгоритм А является модификацией поиска по графу, аналогичного алгоритму Дейкстры, но вместо выбора вершины с минимальным значением $g(n)$ (фактическое расстояние от источника до вершины n) он выбирает вершину с минимальным значением функции $f(n) = g(n) + h(n)$ 96. Здесь $h(n)$ — это эвристическая функция, которая оценивает стоимость кратчайшего пути от вершины n до целевой вершины. Эвристика должна удовлетворять одному важному условию: она должна быть **admissible** (допустимой), то есть никогда не завышать истинную стоимость пути 97. Например, в задаче поиска пути на карте эвристикой $h(n)$ может быть прямолинейное расстояние от n до цели. Если эвристика **admissible**, алгоритм А гарантированно находит оптимальный (кратчайший) путь 98. Более того, если эвристика достаточно информативна (то есть близка к истинной стоимости), А может исследовать значительно меньшее количество вершин по сравнению с Dijkstra's, что делает его гораздо быстрее на практике 114. Например, в задачах искусственного интеллекта А часто показывает ускорение в 1.37 раза по сравнению с Dijkstra's 96. Однако, если эвристика неадекватна или плоха (например, равна нулю), А сводится к алгоритму Дейкстры. Сложность А в худшем случае остается экспоненциальной, но в

среднем для хорошо спроектированных эвристик он демонстрирует отличную производительность. Другим важным свойством является **consistency** (непрерывность), которое гласит, что оценка эвристики для вершины n не должна превышать реальное расстояние от n до её соседа n' плюс оценка эвристики для n' . *Consistent* эвристики также являются *admissible* и обеспечивают дополнительные гарантии для некоторых реализаций A [98](#).

Характеристика	Алгоритм Дейкстры	Алгоритм A*
Цель	Найти кратчайшие пути от одной вершины ко всем остальным 39	Найти кратчайший путь от одной вершины к одной целевой 114
Функция выбора	Минимизация фактического расстояния $g(n)$ 170	Минимизация эвристической функции $f(n) = g(n) + h(n)$ 96
Требования к весам	Неотрицательные веса 150	Неотрицательные веса
Гарантия оптимальности	Да, всегда	Да, при условии, что эвристика $h(n)$ допустима (admissible) 97
Эффективность	Исследует все вершины в порядке возрастания расстояния от источника	Исследует в первую очередь вершины, ведущие к цели, что часто бывает быстрее 96
Временная сложность	$O(E \log V)$ (с бинарной кучей) 205	В худшем случае экспоненциальная, но в среднем лучше, чем у Дейкстры 149
Пространственная сложность	$O(V)$	$O(V)$

Максимальный Поток и Паросочетания: Форд-Фалкерсон, Эдмондс-Карп и Венгерский Алгоритм

Задачи на сетях потоков представляют собой мощный математический аппарат для моделирования и решения широкого круга прикладных проблем, от транспортировки товаров по дорогам до передачи данных в компьютерных сетях [4](#) [111](#). Одной из фундаментальных задач является задача о максимальном потоке, которая заключается в определении максимального количества потока, которое можно отправить из источника (s) в сток (t) в ориентированном взвешенном графе (сети), где вес каждого ребра представляет его пропускную способность [138](#). Метод Форда-Фалкерсона является общей схемой для решения этой задачи. Его основная идея заключается в итеративном увеличении потока. Алгоритм начинает с нулевого потока и на каждом шаге ищет путь увеличения в так называемом остаточном

графе [65](#). Остаточный граф показывает, как можно изменить текущий поток: наличие ребра с положительной остаточной пропускной способностью означает возможность отправить больше потока в прямом направлении, а наличие обратного ребра означает возможность "отозвать" часть потока, отправленного ранее. Путь увеличения — это путь от s до t в остаточном графе, состоящий из ребер с положительной пропускной способностью. Поток увеличивается на максимально возможную величину, равную минимальной остаточной пропускной способности вдоль этого пути. Процесс повторяется до тех пор, пока в остаточном графе не останется ни одного пути увеличения. Корректность этого метода гарантируется теоремой о максимальном потоке и минимальном разрезе, которая утверждает, что максимальный поток равен пропускной способности минимального разреза сети [68](#). Однако, если в качестве алгоритма поиска пути увеличения используется "слепой" поиск (например, DFS), и пропускные способности являются иррациональными числами, метод Форда-Фалкерсона может не сойтись [91](#).

Для обеспечения полиномиального времени выполнения был предложен алгоритм Эдмондса-Карпа, который является конкретной реализацией метода Форда-Фалкерсона [137](#). В этой реализации для поиска пути увеличения всегда используется поиск в ширину [69](#). Это гарантирует, что на каждом шаге будет найден самый короткий путь от s до t в остаточном графе (измеряется в количестве рёбер) [140](#). Такой подход предотвращает "плохие" последовательности увеличений и доказывает, что количество итераций алгоритма не превышает $O(VE)$ [9](#). Поскольку каждая итерация (поиск BFS) занимает $O(E)$ времени, общая временная сложность алгоритма Эдмондса-Карпа составляет $O(VE^2)$ [90](#) [141](#). Этот результат является полиномиальным и делает задачу о максимальном потоке эффективно разрешимой.

Задача о паросочетаниях, в частности, задача о назначениях, является частным случаем задачи о максимальном потоке, но также может быть эффективно решена специализированными алгоритмами. В задаче о назначениях дано n человек и n работ, а также матрица стоимостей $C[i,j]$, где $C[i,j]$ — стоимость того, что i -й человек выполнит j -ю работу. Требуется составить взаимно однозначное соответствие "человек-работа", то есть назначить каждому человеку ровно одну работу, а каждой работе — ровно одного исполнителя, так чтобы суммарная стоимость всех назначений была минимальной. Эту задачу можно свести к задаче о максимальном потоке минимальной стоимости: создаётся двудольный граф, где одна доля состоит из n вершин-людей, другая — из n вершин-работ. Между человеком i и

работой j проводится ребро с пропускной способностью 1 и стоимостью $C[i,j]$. Добавляются источник, соединенный со всеми людьми, и все работы, соединённые со стоком, все новые рёбра имеют пропускную способность 1 и нулевую стоимость. Тогда минимально-стоимостной поток величины n в этой сети соответствует оптимальному назначению [11](#).

Классическим полиномиальным алгоритмом для решения задачи о назначениях является венгерский алгоритм [118](#). Его идея заключается в преобразовании матрицы стоимости таким образом, чтобы найти n нулей, ни один из которых не находился бы в одном ряду или столбце. Эти нули будут соответствовать оптимальному набору назначений с суммарной стоимостью ноль, что эквивалентно минимальной стоимости в исходной задаче. Алгоритм состоит из двух основных этапов, которые повторяются до тех пор, пока не будет найдено оптимальное решение [13](#):

1. Редукция матрицы: Из каждой строки матрицы вычитается её минимальный элемент. Затем из каждого столбца вычитается его минимальный элемент. Это не меняет оптимального решения, но приводит к появлению хотя бы одного нуля в каждой строке и столбце [94](#).
2. Покрытие нулей: Пытаемся покрыть все нули в полученной матрице минимальным количеством горизонтальных и вертикальных линий. Если это количество равно n , значит, найдено n независимых нулей (нулей, не попарно совпадающих по строке или столбцу), и оптимальное решение найдено. В противном случае, из минимального элемента, не покрытого линиями, вычитается из всех элементов, не покрытых линиями, и прибавляется к элементам, лежащим на пересечении линий. Процесс редукции и покрытия повторяется [40](#) [119](#).

Стандартная реализация венгерского алгоритма имеет кубическую временную сложность $O(n^3)$ [78](#), что делает его очень эффективным для решения задач о назначениях.

Задача о Потоке Минимальной Стоимости

Задача о потоке минимальной стоимости является естественным обобщением задачи о максимальном потоке. В этой задаче, помимо пропускных способностей, каждому ребру e сети приписывается стоимость $c(e)$, которую нужно заплатить за единицу потока, проходящего по этому ребру. Цель состоит в том, чтобы найти поток с минимально возможной суммарной стоимостью, который удовлетворяет всем ограничениям по пропускным способностям и

закону сохранения потока (поток, входящий в вершину, равен потоку, выходящему из неё, для всех вершин, кроме источника и стока) [3 111](#). Частным случаем этой задачи является задача о назначениях, а в целом она охватывает огромный класс оптимизационных проблем, включая транспортные задачи, планирование производства и многое другое [62 109](#). Существует несколько классов алгоритмов для решения этой задачи, два самых распространенных из которых — это алгоритмы последовательного поиска кратчайшего пути и алгоритмы отмены циклов [132](#).

Алгоритмы последовательного поиска кратчайшего пути начинаются с нулевого потока и последовательно увеличивают его, отправляя поток по путям минимальной стоимости в остаточном графе. Однако, в отличие от алгоритма Эдмондса-Карпа, который максимизирует объем потока на каждом шаге, здесь на каждом шаге отправляется единица потока по кратчайшему (минимально-стоимостному) пути от источника к стоку в остаточном графе [133](#). Этот процесс повторяется f раз, где f — желаемая величина потока. Если требуется найти поток максимальной величины с минимальной стоимостью (задача о максимальном потоке минимальной стоимости), алгоритм продолжает работать до тех пор, пока существует путь увеличения в остаточном графе. Каждый такой путь будет кратчайшим среди всех увеличивающих путей. Этот метод гарантирует, что найденный поток будет минимально-стоимостным для своей величины. Чтобы найти максимальный поток минимальной стоимости, можно использовать алгоритм, который последовательно отправляет поток по кратчайшим путям, пока это возможно [112](#). Сложность этих алгоритмов зависит от используемого алгоритма для поиска кратчайшего пути (например, Дейкстры или Беллмана-Форда, если есть отрицательные веса в остаточном графе) и от величины максимального потока.

Альтернативный подход основан на алгоритмах отмены циклов. Основная идея заключается в том, что поток является минимально-стоимостным тогда и только тогда, когда в остаточном графе не существует циклов отрицательной стоимости [133](#). Если такой цикл существует, то отправка потока по этому циклу (увеличение потока на ребрах прямого направления и уменьшение на ребрах обратного направления) уменьшит общую стоимость потока без изменения его величины. Таким образом, алгоритм отмены циклов работает следующим образом: он начинает с любого допустимого потока (например, нулевого) и затем итеративно находит в остаточном графе цикл отрицательной стоимости. Если такой цикл найден, поток вдоль него увеличивается на максимально возможную величину (минимальная остаточная пропускная способность вдоль

цикла прямого направления), и процесс повторяется. Если цикла отрицательной стоимости не найдено, текущий поток является минимально-стоимостным ⁸. Эффективность этих алгоритмов зависит от того, как быстро можно найти цикл отрицательной стоимости. Алгоритмы отмены циклов могут быть чувствительны к величине целочисленных параметров сети, таких как пропускные способности и стоимости, что влияет на их полиномиальную оценку сложности ⁸. Существуют различные варианты, отличающиеся стратегией поиска цикла (например, всегда находить цикл с наименьшей средней стоимостью).

Важно отметить, что задача о потоке минимальной стоимости является задачей линейного программирования, и её можно решать с помощью универсальных методов, таких как симплекс-метод. Однако специализированные алгоритмы на графах, такие как вышеописанные, обычно работают гораздо быстрее на графовых структурах ⁶. Кроме того, существуют псевдополиномиальные алгоритмы, основанные на масштабировании пропускных способностей и стоимостей, которые могут быть очень эффективны на практике ¹³⁶. В последние годы были разработаны и алгоритмы, работающие в почти линейном времени, что представляет собой значительный прогресс в данной области ⁴.

Справка

1. [PDF] Algorithm Design and Analysis - CMU School of Computer Science <https://www.cs.cmu.edu/~15451-f25/notes.pdf>
2. Minimum – Peak – Cost Flows Over Time - Wiley Online Library <https://onlinelibrary.wiley.com/doi/full/10.1002/net.70001>
3. Minimum Cost Flow Problem | Springer Nature Link https://link.springer.com/rwe/10.1007/978-0-387-74759-0_373
4. Maximum Flow and Minimum-Cost Flow in Almost-Linear Time <https://dl.acm.org/doi/10.1145/3728631>
5. (PDF) Budget-constrained minimum cost flows - ResearchGate https://www.researchgate.net/publication/277576254_Budget-constrained_minimum_cost_flows

6. [PDF] The Minimum Cost Flow Problem - MIT OpenCourseWare https://ocw.mit.edu/courses/15-053-optimization-methods-in-management-science-spring-2013/687225c151d8cc8254dcb8b07e5f9ce2_MIT15_053S13_lec16.pdf
7. [PDF] Algorithm Design and Analysis - CMU School of Computer Science <https://www.cs.cmu.edu/~15451-s25/notes.pdf>
8. [PDF] A Formal Analysis of Capacity Scaling Algorithms for Minimum-Cost ... <https://arxiv.org/pdf/2602.03701>
9. MIT Courses #7 – Design and Analysis of Algorithms - NikCreate <https://nikcreate.com/2021/03/26/mit-courses-7-design-and-analysis-of-algorithms/>
10. BFS and DFS Algorithm Pseudocode | PDF - Scribd <https://www.scribd.com/document/582499208/Cheatsheet-ASD>
11. [PDF] Lecture: network flow problems <http://faculty.bicmr.pku.edu.cn/~wenzw/bigdata/lect-networksum.pdf>
12. An example of graph G and the minimum cost network flow based ... https://www.researchgate.net/figure/An-example-of-graph-G-and-the-minimum-cost-network-flow-based-solutions_fig2_220462059
13. [PDF] The Hungarian method for the assignment problem <https://web.eecs.umich.edu/~pettie/matching/Kuhn-hungarian-assignment.pdf>
14. Hungarian Algorithm for Assignment Problem (Introduction and ... <https://www.geeksforgeeks.org/dsa/hungarian-algorithm-assignment-problem-set-1-introduction/>
15. Hungarian Algorithm for Assignment Problem | PDF - Scribd <https://www.scribd.com/document/644995170/assignment>
16. hungarian algorithm to solve the assignment problem with the help ... https://www.researchgate.net/publication/372490348_HUNGARIAN_ALGORITHM_TO_SOLVE_THE_ASSIGNMENT_PROBLEM_WITH_THE_HELP_OF_COMPUTATIONAL_TECHNIQUES
17. Depth First Search or DFS for a Graph - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>
18. Breadth First Search or BFS for a Graph - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>
19. Graph Algorithms (BFS, DFS & Shortest Paths) - Kaggle <https://www.kaggle.com/code/ahmedfakhar123/graph-algorithms-bfs-dfs-shortest-paths>
20. [PDF] Algorithms for Nonlinear Models in Computational Finance and their ... https://cs.nyu.edu/media/publications/buff_robert.pdf
21. Time Complexity of BFS and DFS for bot Matrix and Adjacency List <https://stackoverflow.com/questions/68532455/time-complexity-of-bfs-and-dfs-for-bot-matrix-and-adjacency-list>

22. Time and Space Complexity of DFS and BFS Algorithm <https://www.geeksforgeeks.org/dsa/time-and-space-complexity-of-dfs-and-bfs-algorithm/>
23. Comparison between Adjacency List and Adjacency Matrix ... <https://www.geeksforgeeks.org/dsa/comparison-between-adjacency-list-and-adjacency-matrix-representation-of-graph/>
24. Time and Space Complexity of Adjacency Matrix and List - Baeldung <https://www.baeldung.com/cs/adjacency-matrix-list-complexity>
25. [PDF] a shortest path algorithm for real-weighted undirected graphs <https://web.eecs.umich.edu/~pettie/papers/undsp.pdf>
26. [PDF] ADVANCED ALGORITHMS - CMU School of Computer Science <https://www.cs.cmu.edu/~15850/notes/cmu850-f20.pdf>
27. [PDF] Edsger Wybe Dijkstra: His Life, Work, and Legacy - Microsoft <https://www.microsoft.com/en-us/research/wp-content/uploads/2023/03/Lampson-PCS-from-book.pdf>
28. Prim's algorithm using priority_queue in STL - GeeksforGeeks https://www.geeksforgeeks.org/dsa/prims-algorithm-using-priority_queue-stl/
29. Lecture 10: Prim's Algorithm and Priority Queues Study Guide | Quizlet <https://quizlet.com/study-guides/lecture-10-prim-s-algorithm-and-priority-queues-ee2cbaee-545a-48ba-8f8f-00eb702863d2>
30. [PDF] 00463528853593d117000000.pdf - ResearchGate https://www.researchgate.net/profile/Petr_Sosnin/publication/258516985_Interactive_Systems_and_Technologies/links/00463528853593d117000000.pdf
31. New Trends in Databases and Information Systems - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-319-67162-8.pdf>
32. The 51st Annual Meeting of the European Society for Blood and ... <https://www.nature.com/articles/s41409-025-02651-w>
33. Time and Space Complexity Analysis of Kruskal Algorithm <https://www.geeksforgeeks.org/dsa/time-and-space-complexity-analysis-of-kruskal-algorithm/>
34. Kruskal's Minimum Spanning Tree (MST) Algorithm - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/kruskals-minimum-spanning-tree-algorithm-greedy-algo-2/>
35. Kruskal's Algorithm Time Complexity Analysis | PDF - Scribd <https://www.scribd.com/document/56233975/Greedy-Algorithms>
36. [PDF] Kruskal's Minimum Spanning Tree Algorithm & Union-Find Data ... <https://www.cs.cmu.edu/~ckingsf/class/02713-s14/lectures/lec04-othermst.pdf>

37. Kruskal Algorithm: Time Complexity (Example in C - WsCube Tech <https://www.wscubetech.com/resources/dsa/kruskal-algorithm>
38. A Time-Dependent Dijkstra's Algorithm for the Shortest Path ... <https://www.mdpi.com/2079-8954/14/1/61>
39. Functional Correctness of C Implementations of Dijkstra's, Kruskal's ... https://link.springer.com/chapter/10.1007/978-3-030-81688-9_37
40. Hungarian Algorithm Step-by-Step Guide | PDF - Scribd <https://www.scribd.com/doc/36604014/Hungarian-Algorithm>
41. Hungarian Algorithm: finding minimum number of lines to cover ... <https://stackoverflow.com/questions/23379660/hungarian-algorithm-finding-minimum-number-of-lines-to-cover-zeroes>
42. Improved Hungarian algorithm for assignment problems of serial ... https://www.researchgate.net/publication/308911744_Improved_Hungarian_algorithm_for_assignment_problems_of_serial-parallel_systems
43. [PDF] ADVANCED ALGORITHMS - CMU School of Computer Science <http://www.cs.cmu.edu/afs/cs.cmu.edu/academic/class/15850-f20/www/notes/all.pdf>
44. Initial Placement for Fruchterman–Reingold Force Model with ... - arXiv <https://arxiv.org/html/2412.20317v2>
45. Time Complexity of Dijkstra and Kruskal | PDF - Scribd <https://www.scribd.com/presentation/963550473/lec9>
46. Difference between Prim's and Kruskal's algorithm for MST <https://www.geeksforgeeks.org/dsa/difference-between-prims-and-kruskals-algorithm-for-mst/>
47. Graph Algorithms in Data Structure (Time Complexity & Techniques) <https://www.wscubetech.com/resources/dsa/graph-algorithms>
48. 10.5 Searching Algorithms Revisited - Hello Algo https://www.hello-algo.com/en/chapter_searching/searching_algorithm_revisited/
49. New Hash-based Sequence Alignment Algorithm - ACM Digital Library <https://dl.acm.org/doi/fullHtml/10.1145/3523286.3524539>
50. Comparing Linear Search and Binary Search Algorithms to Search ... https://www.researchgate.net/publication/281978797_Comparing_Linear_Search_and_Binary_Search_Algorithms_to_Search_an_Element_from_a_Linear_List_Implemented_through_Static_Array_Dynamic_Array_and_Linked_List
51. [PDF] Algorithm Design and Analysis - CMU School of Computer Science <https://www.cs.cmu.edu/~15451-f24/notes.pdf>

52. Data Structures on Modern Software Development - LinkedIn <https://www.linkedin.com/pulse/data-structures-modern-software-development-pratibha-kumari-jha-7tntc>
53. Understanding Sparse Matrices and Binary Search | Matrix Theory <https://www.scribd.com/document/851472863/BINARY-SEARCH-SPARSE-MATRIX-250320-081458>
54. Video Lectures | Introduction to Algorithms (SMA 5503) https://ocw.mit.edu/courses/6-046j-introduction-to-algorithms-sma-5503-fall-2005/video_galleries/video-lectures/
55. [PDF] 6.046J Complete Lecture Notes - MIT OpenCourseWare https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2012/b1e5ac3c8f7d60db9b8d5b66c40bc55e_MIT6_046JS12_Notes.pdf
56. Problem Session 4 | Introduction to Algorithms - MIT OpenCourseWare <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/problem-session-4/>
57. Readings | Introduction to Algorithms (SMA 5503) <https://ocw.mit.edu/courses/6-046j-introduction-to-algorithms-sma-5503-fall-2005/pages/readings/>
58. Lecture 12: Time Complexity | Theory of Computation | Mathematics <https://ocw.mit.edu/courses/18-404j-theory-of-computation-fall-2020/resources/lecture-12-time-complexity/>
59. Why BFS is better than recursion for tree depth | Satyam Shivhare ... https://www.linkedin.com/posts/satyam-shivhare_leetcode-datastructures-algorithms-activity-7392942819499196417-CyQf
60. Why is the time complexity of both DFS and BFS $O(V + E)$ <https://stackoverflow.com/questions/11468621/why-is-the-time-complexity-of-both-dfs-and-bfs-o-v-e>
61. Data Structures: Stacks and Queues Overview | PDF - Scribd <https://www.scribd.com/document/656281942/CSE601-Data-Structure-and-Algorithm>
62. Time-varying minimum cost flow problems - ScienceDirect.com <https://www.sciencedirect.com/science/article/abs/pii/S037722170000059X>
63. Minimum cost flow problem with conflicts - Şuvak - 2021 - Networks <https://onlinelibrary.wiley.com/doi/10.1002/net.22021>
64. Chapter8 Min Cost Flow | PDF | Mathematical Optimization - Scribd <https://www.scribd.com/document/949897400/Chapter8-Min-Cost-Flow>
65. [PDF] Class on Design and Analysis of Algorithms, Lecture 13 Notes https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/ac6b6e8ac932bb5dfb6506fd0db9d5ad_MIT6_046JS15_lec13.pdf

66. [PDF] Lecture Notes: Introduction to Max Flow 1 Basics 2 Residual Graph ... <https://www.cs.cmu.edu/~15451-f20/LectureNotes/flowIntro.pdf>
67. A feasible flow in a graph and the corresponding residual graph. All... https://www.researchgate.net/figure/A-feasible-flow-in-a-graph-and-the-corresponding-residual-graph-All-edge-capacities-are-3_fig5_220567082
68. Maximum Matching - an overview | ScienceDirect Topics <https://www.sciencedirect.com/topics/computer-science/maximum-matching>
69. Comparative Analysis of Some Methods and Algorithms for Traffic ... <https://www.mdpi.com/2227-7390/13/14/2296>
70. Prim's Algorithm for Minimum Spanning Tree (MST) - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/prims-minimum-spanning-tree-mst-greedy-algo-5/>
71. [PDF] Introduction to Algorithms <https://www.cs.cmu.edu/afs/cs/academic/class/15451-s04/www/Lectures/minimumSpanningTrees.pdf>
72. Space Complexity of Kruskal's Algorithm | PDF | Graph Theory - Scribd <https://www.scribd.com/document/839401914/DAA-Exp5-1>
73. Graph Theory - Kruskal's Algorithm - TutorialsPoint https://www.tutorialspoint.com/graph_theory/graph_theory_kruskals_algorithm.htm
74. Hungarian Algorithm for Bipartite Matching | Matrix (Mathematics) <https://www.scribd.com/document/42532915/The-Hungarian-Algorithm>
75. Hungarian Algorithm for Solving the Assignment Problem - Scribd <https://www.scribd.com/document/272629315/Hungarian-Algorithm-for-Solving-the-Assignment-Problem>
76. Algorithm 1015: A Fast Scalable Solver for the Dense Linear (Sum ... <https://dl.acm.org/doi/fullHtml/10.1145/3442348>
77. [PDF] Proceedings of the 13th International Conference on Computational ... <https://aclanthology.org/W19-57.pdf>
78. Scalability of mass spectrometry-based metabolomics for natural ... <https://pubs.rsc.org/en/content/articlehtml/2026/np/d5np00034c>
79. [PDF] Dynamic Decision Trees and Community-based Graph Embeddings https://theses.hal.science/tel-04956830v1/file/137746_DAMAY_2024_archivage.pdf
80. A Comprehensive Benchmark for Multi-Task Comic Understanding <https://arxiv.org/html/2407.03550v2>
81. Comparing Prim's and Kruskal's Algorithms | PDF - Scribd <https://www.scribd.com/document/551618408/Comparing-Prim-s-With-Kruskal-s-Algo>
82. Topological Sort in Java Explained | PDF - Scribd <https://www.scribd.com/document/848624169/6173e574-d438-45f0-b533-c45f77b7fff8-Untitled>

83. [PDF] MITOCW | Quiz 1 review https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/5148975023959ffe4b43f52277ed8549_e98MPnMHLxE.pdf
84. [PDF] Data structure for representing a graph: combination of linked list ... <https://arxiv.org/pdf/0908.3089>
85. Efficient Binary Search Trees Explained | PDF | Vertex (Graph Theory) <https://www.scribd.com/document/975054815/DS-Module-5-notes>
86. Data Structures and Algorithms:Trees, graphs, and hash tables ... https://www.researchgate.net/publication/387127279_Data_Structures_and_AlgorithmsTrees_graphs_and_hash_tablesSorting_and_searching_algorithms
87. Why are hash table based data structures not the default when ... <https://stackoverflow.com/questions/69777086/why-are-hash-table-based-data-structures-not-the-default-when-implementing-adjac>
88. Implementing Graphs: Edge List, Adjacency List, Adjacency Matrix <https://algodaily.com/lessons/implementing-graphs-edge-list-adjacency-list-adjacency-matrix>
89. Software Testing NPTEL Notes | PDF - Scribd <https://www.scribd.com/document/606580180/Software-Testing-NPTEL-Notes>
90. [PDF] Network Flow and Matching Edmonds-Karp Analysis https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/ba74f89434fc261d9366ebf136c04b52_MIT6_046JS15_Recitation7.pdf
91. [PDF] 6.854J / 18.415J Advanced Algorithms - MIT OpenCourseWare https://ocw.mit.edu/courses/6-854j-advanced-algorithms-fall-2008/10ad543e5ab70c642a82341b3fb0d5f3_lec2.pdf
92. Comparing Kruskal's and Prim's Algorithms - Graph Theory - Scribd <https://www.scribd.com/document/837162227/Kruskal-and-prim-s-algorithm>
93. (PDF) Tutorial on Implementation of Munkres' Assignment Algorithm https://www.researchgate.net/publication/290437481_Tutorial_on_Implementation_of_Munkres'_Assignment_Algorithm
94. Assignment Problem: Hungarian Algorithm | PDF - Scribd <https://www.scribd.com/doc/72286287/Hungarian>
95. [PDF] Variants of the hungarian method for assignment problems <https://web.eecs.umich.edu/~pettie/matching/Kuhn-variants-of-hungarian-assignment.pdf>
96. A*, Bellman-Ford, Dijkstra, AI-Augmented A* and a Neural Baseline <https://www.mdpi.com/2073-431X/14/12/545>
97. Admissibility of A* Algorithm - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/a-is-admissible/>
98. Admissible Heuristic in A* Algorithm | PDF - Scribd <https://www.scribd.com/document/943235390/AI-EXP-6>

99. AND/OR search techniques for chance constrained motion primitive ... <https://www.sciencedirect.com/science/article/abs/pii/S0921889021002505>
100. On Sample-Efficient Generalized Planning via Learned Transition ... <https://arxiv.org/html/2602.23148v1>
101. Dijkstra's Algorithm Step-by-Step Guide | PDF | Graph Theory - Scribd <https://www.scribd.com/doc/7244471/dijkstrasalgorithm>
102. [PDF] Graduate Algorithms, CS580, Spring 2023 Lectures - AWS <https://fas23.s3.amazonaws.com/fas23.pdf>
103. [PDF] On the Comparison-Addition Complexity of All-Pairs Shortest Paths* <https://web.eecs.umich.edu/~pettie/papers/isaac02.pdf>
104. Functional Correctness of C Implementations of Dijkstra's, Kruskal's ... https://www.researchgate.net/publication/353300845_Functional_Correctness_of_C_Implementations_of_Dijkstra's_Kruskal's_and_Prim's_Algorithms
105. Algorithm Analysis in Data Structures | PDF | Time Complexity - Scribd <https://www.scribd.com/document/781021732/DSA-2>
106. [PDF] Advanced Topics in Algorithms - AWS <https://advanced-algorithms-fall-2021.s3.amazonaws.com/aaf21-book.pdf>
107. C Program for DFS and BFS Algorithms | PDF | Graph Theory - Scribd <https://www.scribd.com/document/792902844/Content-Beyond-Syllabus-Subbu-Okk>
108. [PDF] Algorithmic Game Theory - CMU School of Computer Science <https://www.cs.cmu.edu/~sandholm/cs15-892F13/algorithmic-game-theory.pdf>
109. Total fulfillment management: principles, practices and use cases <https://www.sciencedirect.com/science/article/pii/S1366554524004794>
110. Quantum Computing in Logistics and Supply Chain Management <https://arxiv.org/html/2402.17520v1>
111. [PDF] MITx MicroMasters Program in SCM Key Concepts - AWS https://scx-static-assets.s3.amazonaws.com/SCx%20Key%20Concept%20Documents/MITx_MicroMasters_SCM_KeyConcepts.pdf
112. Minimum Cost Maximum Flow from a Graph using Bellman Ford ... <https://www.geeksforgeeks.org/dsa/minimum-cost-maximum-flow-from-a-graph-using-bellman-ford-algorithm/>
113. Kruskal's Algorithm Time Complexity Analysis | PDF - Scribd <https://www.scribd.com/presentation/468329214/Kruskals-Algorithm>
114. A* algorithm and its Heuristic Search Strategy in Artificial Intelligence <https://www.geeksforgeeks.org/artificial-intelligence/a-algorithm-and-its-heuristic-search-strategy-in-artificial-intelligence/>

115. [PDF] Online textbook: Computational Biology: Genomes, Networks ... https://ocw.mit.edu/ans7870/6/6.047/f15/MIT6_047F15_Compiled.pdf
116. [PDF] Data structures - aditya cse <https://adityacse.weebly.com/uploads/2/4/0/7/24078687/data-structures.pdf>
117. LSTM_notebook - Kaggle <https://www.kaggle.com/code/mdumair1/lstm-notebook>
118. Hungarian Method to Solve Assignment Problems - BYJU'S <https://byjus.com/maths/hungarian-method/>
119. Hungarian Algorithm Assignment Solution | PDF - Scribd <https://www.scribd.com/document/878280785/Assignment-Problem-Hungarian-Algorithm>
120. Lecture Notes in Computer Science 2161 - Springer Nature <https://link.springer.com/content/pdf/10.1007/3-540-44676-1.pdf>
121. Data Structures and Algorithms - Narasimha Karumanchi.pdf https://www.academia.edu/38289887/Data_Structures_and_Algorithms_Narasimha_Karumanchi_pdf
122. Download book PDF - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-1-4613-9323-8.pdf>
123. [PDF] Sanskrit Computational Linguistics (Lecture Notes in Artificial ... https://www.researchgate.net/profile/Shrinivasa-Varakhedi/publication/251329647_Asiddhatva_Principle_in_Computational_Model_of_Astadhyayi/links/544b438f0cf2bcc9b1d4f8f9/Asiddhatva-Principle-in-Computational-Model-of-Astadhyayi.pdf
124. DSA Graphs - W3Schools https://www.w3schools.com/dsa/dsa_theory_graphs.php
125. JGraphT—A Java Library for Graph Data Structures and Algorithms <https://dl.acm.org/doi/fullHtml/10.1145/3381449>
126. [PDF] FASTER SCALING ALGORITHMS FOR NETWORK PROBLEMS* <https://web.eecs.umich.edu/~pettie/matching/Gabow-Tarjan-scaling-bipartite-matching.pdf>
127. [PDF] arXiv:1801.09809v2 [cs.DC] 4 Sep 2020 <https://arxiv.org/pdf/1801.09809>
128. Lecture 3: Sets and Sorting | Introduction to Algorithms <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/lecture-3-sets-and-sorting/>
129. Sorting | Foundations of Software Engineering - MIT OpenCourseWare <https://ocw.mit.edu/courses/1-124j-foundations-of-software-engineering-fall-2000/pages/lecture-notes/sorting/>
130. [PDF] 6.006 Introduction to Algorithms, Problem Session 6 Solutions https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/a16d594954dce727798b28bc37d9f6bb_MIT6_006S20_prob6sol.pdf

131. Sorting Algorithms | Introduction to CS and Programming using Python https://ocw.mit.edu/courses/6-100l-introduction-to-cs-and-programming-using-python-fall-2022/resources/6100l-lecture-24-version-2_mp4/
132. [PDF] Minimum Cost Flow <http://www.columbia.edu/~cs2035/courses/ieor6614.S12/mcf.pdf>
133. [PDF] Min Cost Flow - cs.Princeton <https://www.cs.princeton.edu/courses/archive/spr03/cs226/lectures/mincost.4up.pdf>
134. [PDF] Lecture 13 - Network Flow - MIT OpenCourseWare https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2012/7c2927794e61bd70c14c07728fa54375_MIT6_046JS12_lec13.pdf
135. [PDF] Class on Design and Analysis of Algorithms, Lecture 14A Notes https://ocw.mit.edu/courses/6-046j-design-and-analysis-of-algorithms-spring-2015/bdafa05f4f3c20cabfd08d2fbd099f63_MIT6_046JS15_lec14A.pdf
136. Lecture 3 | Advanced Algorithms - MIT OpenCourseWare <https://ocw.mit.edu/courses/6-854j-advanced-algorithms-fall-2008/resources/lec3/>
137. (PDF) Modified EDMONDS-KARP Algorithm to Solve Maximum Flow ... https://www.researchgate.net/publication/297738396_Modified_EDMONDS-KARP_Algorithm_to_Solve_Maximum_Flow_Problems
138. Ford-Fulkerson Algorithm for Maximum Flow Problem - GeeksforGeeks <https://www.geeksforgeeks.org/dsa/ford-fulkerson-algorithm-for-maximum-flow-problem/>
139. Max Flow Algorithm | PDF | Discrete Mathematics - Scribd <https://www.scribd.com/document/176350399/Max-Flow-Algorithm>
140. Complexity Analysis Proof - Mastering the Edmonds-Karp Algorithm <https://oboe.com/learn/mastering-the-edmonds-karp-algorithm-t7z4ky/complexity-analysis-proof-1wnghlq>
141. [PDF] Algorithm Design and Analysis (IX) <https://basics.sjtu.edu.cn/~liguoqiang/teaching/EI6303/lectures/algo9.pdf>
142. How does a Breadth-First Search work when looking for Shortest ... <https://stackoverflow.com/questions/8379785/how-does-a-breadth-first-search-work-when-looking-for-shortest-path>
143. [PDF] Union-Find <https://www.cs.cmu.edu/~15451-f15/lectures/lect0914.pdf>
144. What is better, adjacency lists or adjacency matrices for graph ... <https://stackoverflow.com/questions/2218322/what-is-better-adjacency-lists-or-adjacency-matrices-for-graph-problems-in-c>
145. Introduction to Data Structures - AlgoDaily <https://algodaily.com/lessons/introduction-to-data-structures-38125e53>
146. Understanding Hashing and Hash Tables | PDF - Scribd <https://www.scribd.com/document/912774320/Notes-of-Advanced-Data-Structures>

147. How to think in graphs: An illustrative introduction to Graph Theory ... <https://www.freecodecamp.org/news/i-dont-understand-graph-theory-1c96572a1401/>
148. E-Poster Presentations | European Psychiatry | Cambridge Core <https://www.cambridge.org/core/journals/european-psychiatry/article/eposter-presentations/19AB1C31ED973A89A1800893F0F298D5>
149. Lecture Notes in Artificial Intelligence 2371 - Springer <https://link.springer.com/content/pdf/10.1007/3-540-45622-8.pdf>
150. Dijkstra's Algorithm: Step-by-Step Guide | PDF - Scribd <https://www.scribd.com/document/836813989/62837Lecture-04-b-Dijkstras-Algorithm>
151. Advances in Information Technology and Education - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-642-22418-8.pdf>
152. Parallel Shortest Path Algorithms for Solving Large-Scale Instances https://www.academia.edu/89064679/Parallel_Shortest_Path_Algorithms_for_Solving_Large_Scale_Instances
153. Unlock your Introduction to Algorithms 4th Edition by Thomas ... <https://pt.scribd.com/document/1021768798/Unlock-your-Introduction-to-Algorithms-4th-Edition-by-Thomas-Cormen-Charles-Leiserson-Ronald-Rivest-Clifford-Stein-ISBN-0262367505-9780262367509-di>
154. Frontier and Future Development of Information Technology in ... <https://link.springer.com/content/pdf/10.1007/978-94-007-7618-0.pdf>
155. [PDF] Modified EDMONDS-KARP Algorithm to Solve Maximum Flow ... https://www.scirp.org/pdf/ojapps_2016022916424713.pdf
156. 克鲁斯卡尔算法 - 稀土掘金 <https://juejin.cn/post/7618497511616708660>
157. Kruskal's Spanning Tree Algorithm - iT 邦幫忙 <https://ithelp.ithome.com.tw/articles/10239656>
158. Does using union-find in Kruskal's algorithm actually affect worst ... <https://stackoverflow.com/questions/32040718/does-using-union-find-in-kruskals-algorithm-actually-affect-worst-case-runtime>
159. [PDF] dijkstra's algorithm with predictions to solve the single-source many ... <https://arxiv.org/pdf/2112.11927>
160. A* Admissible Heuristic for die rolling on grid - Stack Overflow <https://stackoverflow.com/questions/16547724/a-admissible-heuristic-for-die-rolling-on-grid>
161. Hybrid Path Planning Algorithm for Autonomous Mobile Robots - MDPI <https://www.mdpi.com/2224-2708/14/5/87>
162. Sustainability, Volume 15, Issue 14 (July-2 2023) – 731 articles - MDPI <https://www.mdpi.com/2071-1050/15/14?page=2>

163. [PDF] IAEA TECDOC SERIES https://www-pub.iaea.org/MTCD/Publications/PDF/TE-1935_web.pdf
164. [PDF] Network Flows II - Carnegie Mellon University <https://www.cs.cmu.edu/~15451-f24/lectures/lecture12-flow2.pdf>
165. Analysis and Design of Algorithm Lecture Notes | PDF - Scribd <https://www.scribd.com/document/343694283/Analysis-and-Design-of-Algorithm-Lecture-notes>
166. cracking the coding interview 189 programming questions & solutions https://www.academia.edu/43135882/CRACKING_THE_CODING_INTERVIEW_189_PROGRAMMING_QUESTIONS_and_SOLUTIONS
167. Basic graph algorithms <https://dl.acm.org/doi/pdf/10.5555/1882757.1882764>
168. Unit 9final | PDF | Time Complexity - Scribd <https://www.scribd.com/document/947414220/Unit-9final>
169. Shruti Pokale's Post - Graphs 101: Kruskal's Algorithm - LinkedIn https://www.linkedin.com/posts/shruti-pokale_graphs-101-kruskals-algorithm-activity-7429034338521989121_QrW
170. Dijkstra's Algorithm Overview and Steps | PDF - Scribd <https://www.scribd.com/document/434601666/Dijkstra-s-algorithm-pdf>
171. Using Machine Learning Predictions to Speed-up Dijkstra's Shortest ... https://www.researchgate.net/publication/357267413_Using_Machine_Learning_Predictions_to_Speed-up_Dijkstra's_Shortest_Path_Algorithm
172. Introduction to Algorithms, fourth edition - Amazon.com <https://www.amazon.com/Introduction-Algorithms-fourth-Thomas-Cormen/dp/026204630X>
173. Algorithm Exercises & Analysis | PDF | Combinatorics - Scribd <https://www.scribd.com/document/685139077/CLRS-Solutions>
174. Asymptotic Analysis of Algorithms | PDF | Time Complexity - Scribd <https://www.scribd.com/document/408631767/CLRS-Solutions-Manual-pdf>
175. DiEC: Diffusion Embedded Clustering - arXiv <https://arxiv.org/html/2512.20905v1>
176. [PDF] Multiple point clouds registration with deep learning https://theses.hal.science/tel-05513315v1/file/VEDRENNE_Luc_2025_ED269.pdf
177. Non-recursive Depth-First Search (DFS) Using a Stack <https://stackoverflow.com/questions/10342306/non-recursive-depth-first-search-dfs-using-a-stack>
178. Detecting a cycle in a directed graph using DFS? - Stack Overflow <https://stackoverflow.com/questions/31542031/detecting-a-cycle-in-a-directed-graph-using-dfs>

179. Graph Algorithm To Find All Connections Between Two Arbitrary ... <https://stackoverflow.com/questions/58306/graph-algorithm-to-find-all-connections-between-two-arbitrary-vertices>
180. Recent Advances in Fully Dynamic Graph Algorithms <https://dl.acm.org/doi/full/10.1145/3555806>
181. Algorithm Design and Analysis Techniques | PDF | Time Complexity <https://www.scribd.com/document/865725912/daa-ai>
182. Study Guide - cs.Princeton <https://www.cs.princeton.edu/courses/archive/spr13/cos226/studyGuide.html>
183. CS Data Structures Exam 1 Flashcards - Quizlet <https://quizlet.com/446854813/cs-data-structures-exam-1-flash-cards/>
184. Data Structures: Hashing & Trees | PDF | Vertex (Graph Theory) <https://www.scribd.com/document/400077495/COP3530-Cheat-Sheet-data-structures>
185. Graph Search Problem - an overview | ScienceDirect Topics <https://www.sciencedirect.com/topics/computer-science/graph-search-problem>
186. Theoretically Efficient Parallel Graph Algorithms Can Be Fast and ... <https://dl.acm.org/doi/fullHtml/10.1145/3434393>
187. A Framework for LLM Reasoning Across All Domains - LinkedIn https://www.linkedin.com/posts/wenhu-chen-ab59317b_general-reasoner-generalizing-llm-reasoning-activity-7318016964570882048-AjMc
188. Overview of Searching Algorithms | PDF | Time Complexity - Scribd <https://www.scribd.com/document/883339736/Searching-Algorithms>
189. Internet and Modern Society - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-032-05144-8.pdf>
190. [PDF] Frontiers in Software Engineering Education - ResearchGate https://www.researchgate.net/profile/Manuel-Mazzara/publication/343592688_Towards_Code_Review_Guideline_in_a_Classroom/links/5f6683ffa6fdcc00862ee84c/Towards-Code-Review-Guideline-in-a-Classroom.pdf?origin=scientificContributions
191. Sergey Kovalev Igor Kotenko Andrey Sukhanov Editors <https://link.springer.com/content/pdf/10.1007/978-3-031-43789-2.pdf>
192. [PDF] Proceedings of Seventh International Congress on Information and ... https://www.researchgate.net/profile/Makinde-Ajayi-2/publication/362453203_Promoting_Viable_Supply_Chain_Management_SCM_in_the_Nigeria_Agro-Allied_Industry_Using_Internet_of_Things/links/631b8a6a873eca0c00752deb/Promoting-Viable-Supply-Chain-Management-SCM-in-the-Nigeria-Agro-Allied-Industry-Using-Internet-of-Things.pdf

193. Software Engineering Methods in Systems and Network Systems <https://link.springer.com/content/pdf/10.1007/978-3-031-54813-0.pdf>
194. LAO*: A heuristic search algorithm that finds solutions with loops https://www.researchgate.net/publication/222544473_LAO_A_heuristic_search_algorithm_that_finds_solutions_with_loops
195. PACMPL: Vol 7, No POPL - ACM Digital Library <https://dl.acm.org/toc/pacmpl/2023/7/POPL?tocHeading=heading1>
196. Prim's algorithm with Fibonacci heap: why $O(E + V \cdot \log(V))$? <https://stackoverflow.com/questions/51264504/prims-algorithm-with-fibonacci-heap-why-oe-vlogv>
197. Time complexity analysis of Kruskal's Algorithm <https://math.stackexchange.com/questions/4769651/time-complexity-analysis-of-kruskals-algorithm>
198. how to calculate the time complexity of kruskal's algorithm which is ... <https://stackoverflow.com/questions/23057376/how-to-calculate-the-time-complexity-of-kruskals-algorithm-which-is-may-be-oe>
199. ARA*: Anytime A* with Provable Bounds on Sub-Optimality https://www.researchgate.net/publication/221619552_ARA_Anytime_A_with_Provable_Bounds_on_Sub-Optimality
200. Dijkstra's Algorithm: Step-by-Step Guide | PDF - Scribd <https://fr.scribd.com/document/836813989/62837Lecture-04-b-Dijkstras-Algorithm>
201. [PDF] Online textbook: Computational Biology: Genomes, Networks ... https://ocw.mit.edu/courses/6-047-computational-biology-fall-2015/mit6_047f15_compiled.pdf
202. Introduction to Algorithms, 3rd Edition - Amazon.com <https://www.amazon.com/Introduction-Algorithms-3rd-MIT-Press/dp/0262033844>
203. Bellman Ford's Algorithm - Programiz <https://www.programiz.com/dsa/bellman-ford-algorithm>
204. DSA Dijkstra's Algorithm - W3Schools https://www.w3schools.com/dsa/dsa_algo_graphs_dijkstra.php
205. Dijkstra Algorithm: Time Complexity (Example in C - WsCube Tech <https://www.wscubetech.com/resources/dsa/dijkstra-algorithm>
206. Dijkstra's Algorithm - Programiz <https://www.programiz.com/dsa/dijkstra-algorithm>
207. Prim's Algorithm: Time Complexity (Example in C/ C++ - WsCube Tech <https://www.wscubetech.com/resources/dsa/prim-algorithm>
208. (DOC) distributed-graph-algorithms-for-computer-networks (1).docx https://www.academia.edu/38348890/distributed_graph_algorithms_for_computer_networks_1_docx

209. Download book PDF - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-1-4757-3145-3.pdf>
210. Download book PDF - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-1-4419-0534-5.pdf>